

.ekbf File Format Specification

Revision 1.0 — Architecture Reference

1 Global File Layout

The **.ekbf** format is a packed binary file layout optimized for fast memory-mapped reads ($O(1)$ lookup times) and compact storage of symmetric/lower-triangular numerical data structures.

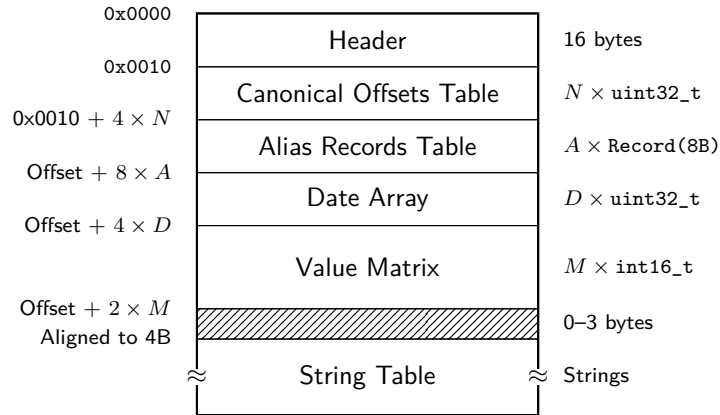


Figure 1: .ekbf Top-Level Memory Layout Structure

1.1 Overview & Section Definitions

The file consists of a fixed 16-byte Header followed by six contiguous payload sections. All multibyte integer primitives are stored in ****Little-Endian**** byte ordering. The String Table section is guaranteed to begin on a ****4-byte alignment boundary****; if necessary, 1 to 3 null padding bytes (0x00) follow the Value Matrix.

Section	Element Type	Total Size	Description
Header	Composite	16 Bytes	Fixed file signature and metadata counters.
Canonical Offsets	<code>uint32_t</code>	$4 \times N$ Bytes	Offsets into String Table for canonical entity names.
Alias Records	Struct (8B)	$8 \times A$ Bytes	ID mapping schema pairing aliases to target IDs.
Date Array	<code>uint32_t</code>	$4 \times D$ Bytes	Numerical dates (YYYYMMDD), one per unique release date.
Value Matrix	<code>int16_t</code>	$2 \times M$ Bytes	Packed lower triangular data matrix ($M = \frac{N(N+1)}{2}$).
Alignment Padding	Byte array	0 to 3 Bytes	Null-byte padding (0x00) aligning String Table to 4B.
String Table	C-Strings	Variable	Null-terminated (<code>\0</code>) UTF-8 encoded string block.

Table 1: Summary of .ekbf File Sections

2 Section Specifications

2.1 Header Layout

The file header is exactly 16 bytes. It defines the Magic Number identifier, format version, and element counts (N, A, D, M) necessary to compute section offsets programmatically.

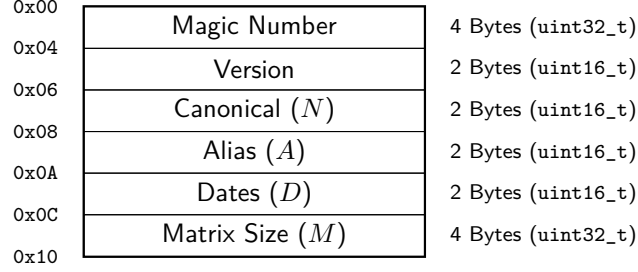


Figure 2: Header Structure Offset Diagram

Offset	Field	Type	Size	Description
0x00	Magic Number	uint32_t	4 Bytes	Unique binary signature for file identification.
0x04	Version	uint16_t	2 Bytes	Format specification version number.
0x06	Canonical (N)	uint16_t	2 Bytes	Total number of canonical offset elements; also defines the Value Matrix row/column dimension.
0x08	Alias (A)	uint16_t	2 Bytes	Total count of alias records.
0x0A	Dates (D)	uint16_t	2 Bytes	Total number of unique dates in the Date Array.
0x0C	Matrix Size (M)	uint32_t	4 Bytes	Total packed elements: $M = \frac{N(N+1)}{2}$.

Table 2: Detailed Header Field Specifications

2.2 Canonical Offsets Table

The Canonical Offsets Table is an array of N contiguous `uint32_t` values starting immediately after the Header at byte offset `0x0010`. Each element holds an absolute byte offset pointing into the String Table where the entity’s null-terminated name string begins. An entity’s position in this array (its *canonical ID*, $0 \dots N - 1$) is what the Value Matrix is indexed by.

2.3 Alias Records Table

The Alias Records Table contains A consecutive 8-byte structures. Each record provides a mapping between an alias entity and its target canonical entity.

Struct Offset	Field	Type	Size	Description
0x00	Alias ID	uint32_t	4 Bytes	Offset into String Table for alias string.
0x04	Target Canonical ID	uint32_t	4 Bytes	Index matching entry in Canonical Offsets Table.

Table 3: Alias Record Structure Schema

2.4 Date Array

The Date Array contains D values encoded as 32-bit unsigned integers (`uint32_t`). Each element stores a numerical representation of a calendar date formatted as `YYYYMMDD` (e.g., `20260726` for July 26, 2026).

Dates are not directly indexed by Value Matrix row/column position; a matrix cell instead references a specific date by index, as described in Section 2.5.1.

2.5 Value Matrix

To maximize space efficiency for symmetric matrices, the format stores only the ****lower triangular**** elements (including the diagonal). The 2D matrix structure is flattened and packed into a contiguous 1D array of M signed 16-bit integers (`int16_t`), where $M = \frac{N(N+1)}{2}$.

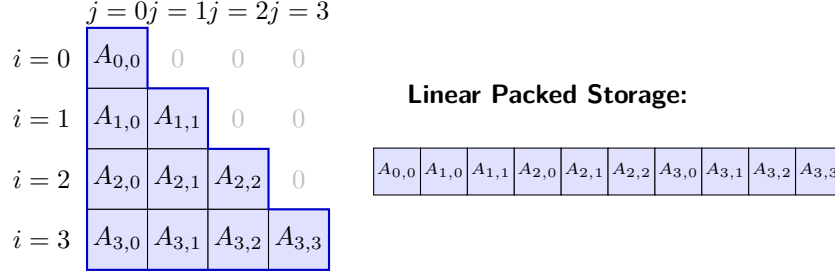


Figure 3: Lower Triangular Matrix Mapping to 1D Array (illustrative 4×4 example; in the real format, the dimension is N , not 4)

Lower Triangular Index Computation

For a matrix with N row/column dimensions, any entry $A_{i,j}$ (where i is row index and j is column index, with $0 \leq j \leq i < N$) is mapped to its linear array index $\text{index}(i, j)$ using the formula:

$$\text{index}(i, j) = \frac{i(i+1)}{2} + j$$

For queries in the upper triangle ($j > i$), exploit symmetry by swapping indices: $\text{index}(j, i)$.

2.5.1 Cell Value Encoding

A matrix cell value of 0 indicates the pair (i, j) has no known combination. A non-zero cell encodes both a date reference and an orientation flag, packed into a single signed `int16_t`:

- **Magnitude** ($|\text{value}|$) is one plus the index into the Date Array: $\text{dateIndex} = |\text{value}| - 1$.
- **Sign** indicates which of the two operands is the “primary” one used as the directory component of the resulting asset URL: a *positive* value means the row operand (i) is primary; a *negative* value means the column operand (j) is primary.

Because one bit of range is spent on sign rather than magnitude, the maximum usable magnitude is 32767; the value -32768 is never written, as it has no positive counterpart. This caps the format at 32767 unique dates.

2.6 String Table

The String Table contains a sequential sequence of variable-length UTF-8 encoded strings. Every string must end with a null terminator byte (`0x00`). Strings are referenced directly via byte offsets from other sections.